

Database Design Project – Phase 2

Conceptual / Logical Database Design

Project Title:

Multi-Tenant LLM API Gateway with Billing, Quota, and Audit

Team	Info
Group	abc
Member 1	runxin shao
Member 2	ziqu yang

Abstract

This Phase 2 document presents the full conceptual and logical database design for the Multi-Tenant LLM API Gateway system. Building on the problem definition, queries, and assumptions from Phase 1, this report provides: (1) an Enhanced Entity-Relationship (EER) diagram capturing all entities, relationships, weak entities, and specializations; (2) a complete relational schema derived from the EER model with primary keys, foreign keys, and candidate keys identified; and (3) relational algebra expressions for the 20 queries defined in Phase 1. The design prioritizes referential integrity, auditability, and transactional quota enforcement across a multi-tenant environment.

Step 5: Enhanced Entity-Relationship (EER) Diagram

5.1 Overview of Entities and Relationships

The EER diagram below models the Multi-Tenant LLM API Gateway database. The design includes 11 entities (2 of which are weak), 10+ relationships of varying cardinalities, relationship attributes, and a disjoint generalization/specialization on ApiKey.

Entities Summary

Entity	Type	Description
Tenant	Regular	An organization or team that owns one or more projects. Identified by tenant_id.
Project	Regular	A logical grouping under a tenant (e.g., dev/prod). Belongs to exactly one tenant.
ApiKey	Regular	A credential key issued to a project. Can be active or revoked. Has created_at, key_hash.
LlmModel	Regular	An LLM model in the catalog (provider, name, version). Referenced by requests.
ModelPricing	Regular	Pricing configuration for a model for a billing month (input_rate, output_rate per 1K tokens).
Request	Regular	A single API call routed through the gateway. Stores input_tokens, status, idempotency_key.
Response	Weak	The response record for a request. Identified by (request_id, response_id). Stores output_tokens, latency_ms, error_type.
DeniedEvent	Weak	A rate-limit or quota-denial event. Identified by (request_id, event_id). Stores reason, denied_at.
MonthlyQuota	Regular	Quota definition per project per month. Stores token_limit, tokens_used, billing_month.
Invoice	Regular	A monthly invoice for a project. Stores total_cost, billing_month, issued_at.
AuditLog	Regular	An audit trail entry referencing a request, key, and action type (e.g., REVOKE, QUOTA_EXCEEDED).

Relationships Summary

Relationship	Entity 1	Entity 2	Notes
owns	Tenant (1)	Project (N)	A tenant owns many projects; each project belongs to one tenant.
has_key	Project (1)	ApiKey (N)	A project can have many API keys; each key belongs to one project.
submits	ApiKey (1)	Request (N)	An API key submits many requests; each request is tied to one key.
uses_model	Request (N)	LlmModel (1)	Each request targets one model; models handle many requests. Attribute: computed_cost.
generates	Request (1)	Response (0..1)	Identifying relationship. Each request produces at most one response.

triggers_denied	Request (1)	DeniedEvent (N)	Identifying relationship. A request can produce multiple denied events.
quota_def	Project (1)	MonthlyQuota (N)	A project has a quota record per month.
billed	Project (1)	Invoice (N)	A project can have multiple monthly invoices.
priced_as	LlmModel (1)	ModelPricing (N)	A model can have different pricing per billing month.
audited_by	Request (1)	AuditLog (N)	A request can appear in multiple audit log entries.

Generalization / Specialization

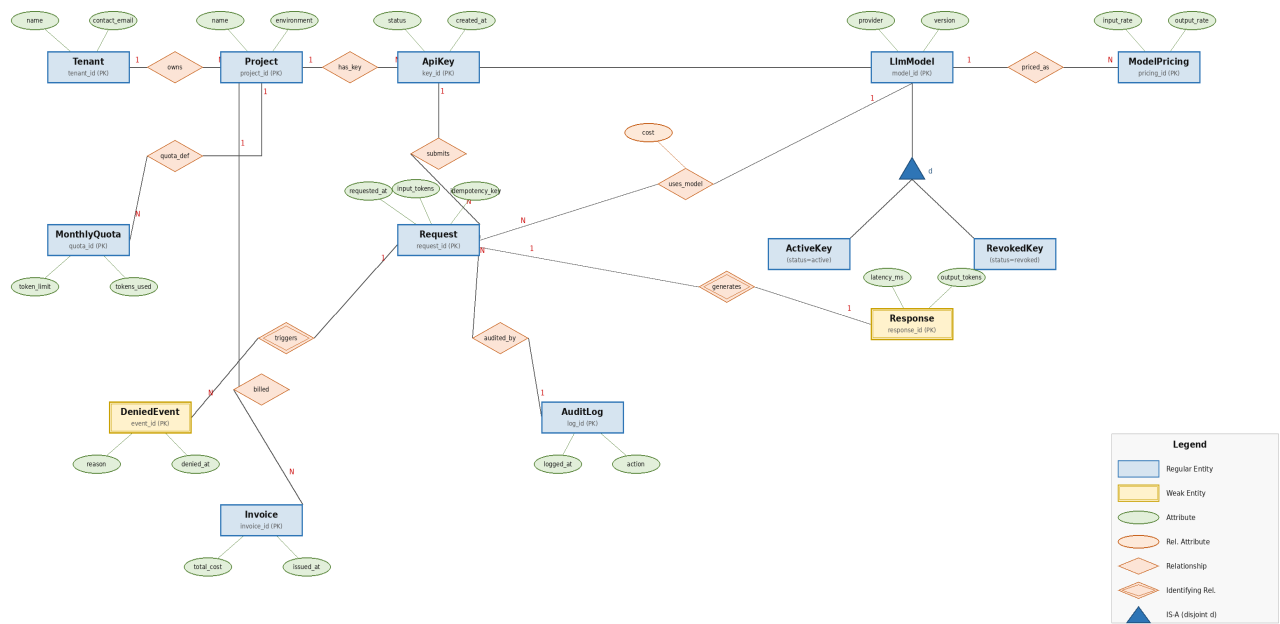
ApiKey is specialized into two disjoint subtypes based on the status attribute:

- ActiveKey – an API key that is currently valid and can be used to submit requests.
- RevokedKey – an API key that has been revoked and should no longer accept requests. Stores revoked_at timestamp.

The specialization is disjoint (d): an ApiKey instance belongs to exactly one subtype at a time. It is partial/total (every key is eventually one of the two states).

5.2 EER Diagram

The following EER diagram uses the notation taught in lectures: rectangles for entities, double-bordered rectangles for weak entities, diamonds for relationships, double-bordered diamonds for identifying relationships, ellipses for attributes, and a filled triangle for IS-A (generalization).



EER Diagram – Multi-Tenant LLM API Gateway Database

Figure 1: EER Diagram – Multi-Tenant LLM API Gateway

Step 6: Creating Relations (Relational Schema)

6.1 Mapping Rules Applied

The following mapping rules from the relational model were applied to convert the EER diagram into relations:

- Each regular entity becomes a relation with all its attributes. The entity key becomes the primary key.
- Each weak entity becomes a relation; its primary key is a composite of the owner entity PK plus the weak entity's partial key.
- Each 1:N relationship is mapped by adding the PK of the '1' side as a foreign key in the 'N' side relation.
- Each 1:1 identifying relationship (Response) is folded into the weak entity relation.
- The generalization/specialization (ApiKey → ActiveKey, RevokedKey) is mapped using the single-table-with-type-discriminator approach, with subtype attributes added as nullable columns.
- Relationship attributes (computed_cost on uses_model) are included in the Request relation.

6.2 Relational Schema

R1. Tenant

Attribute	Type	Constraint	Notes
tenant_id	INT	PK	Surrogate primary key
name	VARCHAR(100)	UNIQUE, NOT NULL	Candidate key – tenant names are unique
contact_email	VARCHAR(200)	NOT NULL	
created_at	DATETIME	NOT NULL	Default CURRENT_TIMESTAMP
status	ENUM('active','suspended')	NOT NULL	

Functional Dependencies: tenant_id → name, contact_email, created_at, status; name → tenant_id (candidate key)

R2. Project

Attribute	Type	Constraint	Notes
project_id	INT	PK	Surrogate primary key
tenant_id	INT	FK → Tenant, NOT NULL	Maps owns relationship (1:N)
name	VARCHAR(100)	NOT NULL	
environment	ENUM('dev','staging','prod')	NOT NULL	
created_at	DATETIME	NOT NULL	

Candidate Key: (tenant_id, name) – project names are unique within a tenant.

Functional Dependencies: project_id → tenant_id, name, environment, created_at; (tenant_id, name) → project_id

R3. ApiKey

Attribute	Type	Constraint	Notes
key_id	INT	PK	Surrogate primary key
project_id	INT	FK → Project, NOT NULL	Maps has_key (1:N)
key_hash	VARCHAR(64)	UNIQUE, NOT NULL	SHA-256 hash of the raw key; candidate key
status	ENUM('active','revoked')	NOT NULL	Discriminator for subtype
created_at	DATETIME	NOT NULL	
revoked_at	DATETIME	NULL	Subtype attribute for RevokedKey
label	VARCHAR(100)	NULL	Human-readable name

Candidate Key: key_hash (globally unique API key string).

Functional Dependencies: key_id → project_id, key_hash, status, created_at, revoked_at, label;
key_hash → key_id

R4. LlmModel

Attribute	Type	Constraint	Notes
model_id	INT	PK	
provider	VARCHAR(50)	NOT NULL	e.g., OpenAI, Anthropic, Mistral
model_name	VARCHAR(100)	NOT NULL	e.g., gpt-4o, claude-3-5-sonnet
version	VARCHAR(50)	NOT NULL	
is_active	BOOLEAN	NOT NULL	

Candidate Key: (provider, model_name, version) – uniquely identifies a model variant.

Functional Dependencies: model_id → provider, model_name, version, is_active; (provider, model_name, version) → model_id

R5. ModelPricing

Attribute	Type	Constraint	Notes
pricing_id	INT	PK	
model_id	INT	FK → LlmModel, NOT NULL	Maps priced_as (1:N)
billing_month	CHAR(7)	NOT NULL	Format: YYYY-MM
input_rate	DECIMAL(10,6)	NOT NULL	Cost per 1K input tokens
output_rate	DECIMAL(10,6)	NOT NULL	Cost per 1K output tokens

Candidate Key: (model_id, billing_month) – one pricing row per model per month.

Functional Dependencies: pricing_id → model_id, billing_month, input_rate, output_rate; (model_id, billing_month) → pricing_id, input_rate, output_rate

R6. Request

Attribute	Type	Constraint	Notes
request_id	INT	PK	
key_id	INT	FK → ApiKey, NOT NULL	Maps submits (1:N)
model_id	INT	FK → LlmModel, NOT NULL	Maps uses_model (N:1)
project_id	INT	FK → Project, NOT NULL	Denormalized for efficient queries
requested_at	DATETIME	NOT NULL	
input_tokens	INT	NOT NULL	
status	ENUM('success','failed','denied')	NOT NULL	
idempotency_key	VARCHAR(128)	NULL	
computed_cost	DECIMAL(12,6)	NULL	Relationship attribute from uses_model
environment	ENUM('dev','staging','prod')	NOT NULL	Copied from project at request time

Candidate Key: (project_id, idempotency_key) – within a project, idempotency_key must be unique.

Functional Dependencies: request_id → key_id, model_id, project_id, requested_at, input_tokens, status, idempotency_key, computed_cost, environment

R7. Response (Weak Entity)

Attribute	Type	Constraint	Notes
response_id	INT	PK	Surrogate; partial key is request_id
request_id	INT	FK → Request, UNIQUE, NOT NULL	Identifying relationship (1:1)
output_tokens	INT	NULL	
latency_ms	INT	NULL	
http_status	INT	NULL	e.g., 200, 400, 500
error_type	VARCHAR(100)	NULL	e.g., TOKEN_LIMIT, AUTH_ERROR
raw_response	TEXT	NULL	Truncated response body
responded_at	DATETIME	NOT NULL	

Functional Dependencies: response_id → request_id, output_tokens, latency_ms, http_status, error_type, responded_at; request_id → response_id (1:1 constraint enforced by UNIQUE)

R8. DeniedEvent (Weak Entity)

Attribute	Type	Constraint	Notes
event_id	INT	PK	Surrogate; partial key is request_id + sequence
request_id	INT	FK → Request, NOT NULL	Identifying relationship
reason	ENUM('QUOTA_EXCEEDED','RATE_LIMITED','KEY_REVOKED')	NOT NULL	
denied_at	DATETIME	NOT NULL	
threshold_pct	DECIMAL(5,2)	NULL	Quota usage % at time of denial

Functional Dependencies: event_id → request_id, reason, denied_at, threshold_pct

R9. MonthlyQuota

Attribute	Type	Constraint	Notes
quota_id	INT	PK	
project_id	INT	FK → Project, NOT NULL	Maps quota_def (1:N)
billing_month	CHAR(7)	NOT NULL	YYYY-MM
token_limit	BIGINT	NOT NULL	
tokens_used	BIGINT	NOT NULL DEFAULT 0	Updated transactionally
cost_limit	DECIMAL(12,2)	NULL	Optional monetary cap

Candidate Key: (project_id, billing_month) – one quota record per project per month.

Functional Dependencies: quota_id → project_id, billing_month, token_limit, tokens_used, cost_limit;
(project_id, billing_month) → quota_id

R10. Invoice

Attribute	Type	Constraint	Notes
invoice_id	INT	PK	
project_id	INT	FK → Project, NOT NULL	Maps billed (1:N)
billing_month	CHAR(7)	NOT NULL	

total_cost	DECIMAL(12,4)	NOT NULL	
total_tokens	BIGINT	NOT NULL	
issued_at	DATETIME	NOT NULL	
paid	BOOLEAN	NOT NULL DEFAULT FALSE	

Candidate Key: (project_id, billing_month) – one invoice per project per month.

Functional Dependencies: invoice_id → project_id, billing_month, total_cost, total_tokens, issued_at, paid

R11. AuditLog

Attribute	Type	Constraint	Notes
log_id	INT	PK	
request_id	INT	FK → Request, NULL	Maps audited_by (N:1); NULL if admin action
key_id	INT	FK → ApiKey, NULL	
action	VARCHAR(100)	NOT NULL	e.g., REQUEST_DENIED, KEY_REVOKED
performed_by	VARCHAR(100)	NOT NULL	system or admin username
logged_at	DATETIME	NOT NULL	
details	TEXT	NULL	JSON blob for extra context

Functional Dependencies: log_id → request_id, key_id, action, performed_by, logged_at, details

6.3 Relational Schema Diagram (Text Notation)

The complete schema in standard notation (PK underlined, FK denoted by →):

```
Tenant (tenant_id, name, contact_email, created_at, status)
Project (project_id, tenant_id → Tenant, name, environment, created_at)
ApiKey (key_id, project_id → Project, key_hash, status, created_at, revoked_at, label)
LlmModel (model_id, provider, model_name, version, is_active)
ModelPricing (pricing_id, model_id → LlmModel, billing_month, input_rate, output_rate)
Request (request_id, key_id → ApiKey, model_id → LlmModel, project_id → Project,
         requested_at, input_tokens, status, idempotency_key, computed_cost,
         environment)
Response (response_id, request_id → Request [UNIQUE], output_tokens, latency_ms,
         http_status, error_type, raw_response, responded_at)
DeniedEvent (event_id, request_id → Request, reason, denied_at, threshold_pct)
MonthlyQuota (quota_id, project_id → Project, billing_month, token_limit, tokens_used,
              cost_limit)
Invoice (invoice_id, project_id → Project, billing_month, total_cost, total_tokens,
         issued_at, paid)
```

```
AuditLog (log_id, request_id → Request, key_id → ApiKey, action, performed_by,  
logged_at, details)
```

Step 7: Relational Algebra Expressions

The following relational algebra expressions correspond to the 20 queries defined in Phase 1 (Step 3).
Notation used:

- σ (sigma) – Selection
- π (pi) – Projection
- $\bowtie$ (bowtie) – Natural Join / Join with condition
- ρ (rho) – Rename
- γ (gamma) – Grouping and aggregation (extended RA)
- $\cup, \cap, -$ – Set union, intersection, difference
- $\times$ – Cartesian product

Q1 – Total cost and tokens per project (last 30 days)

For each project, show total cost and total tokens (input + output) over the last 30 days.

```
T1 ← σ(requested_at ≥ NOW()-30days) (Request)
T2 ← T1 ⋈(Request.request_id = Response.request_id) Response
T3 ← T2 ⋈(T2.project_id = Project.project_id) Project
Result ← γ project_id, project.name;
        SUM(computed_cost)→total_cost,
        SUM(input_tokens + output_tokens)→total_tokens (T3)
```

Q2 – Top 5 most expensive projects per tenant (monthly)

Aggregate monthly cost by tenant and list the top 5 most expensive projects per tenant.

```
T1 ← γ project_id; SUM(computed_cost)→monthly_cost (Request)
T2 ← T1 ⋈ Project ⋈ Tenant
T3 ← γ tenant_id, project_id; monthly_cost, RANK()OVER(PARTITION BY tenant_id
        ORDER BY monthly_cost DESC)→rank (T2)
Result ← π tenant_id, project_id, monthly_cost (σ rank ≤ 5 (T3))
```

Q3 – Success rate, avg latency, failure rate per model (last month)

For each model, show success rate, average latency, and failure rate over the last month.

```
T1 ← σ(requested_at IN current_month) (Request)
T2 ← T1 ⋈(T1.request_id = Response.request_id) Response
Result ← γ model_id;
        COUNT(*)→total,
        SUM(status='success')/COUNT(*)→success_rate,
        SUM(status='failed')/COUNT(*)→failure_rate,
        AVG(latency_ms)→avg_latency (T2)
```

Q4 – Failed requests by error_type per model and project

Breakdown failed requests by error_type per model and per project.

```
T1 ← σ(status = 'failed') (Request)
```

```
T2 ← T1 ⋈(T1.request_id = Response.request_id) Response
Result ← γ model_id, project_id, error_type; COUNT(*)→fail_count (T2)
```

Q5 – Anomaly detection: projects with 24h volume >> 7-day average

Detect projects with request volume in the last 24 hours significantly higher than their 7-day average.

```
Daily ← γ project_id; COUNT(*)→cnt_24h
        (σ requested_at ≥ NOW()-1day (Request))
Weekly ← γ project_id; COUNT(*)/7→avg_daily_7d
        (σ requested_at ≥ NOW()-7days (Request))
T1 ← Daily ⋈(Daily.project_id = Weekly.project_id) Weekly
Result ← π project_id, cnt_24h, avg_daily_7d
        (σ cnt_24h > 2 * avg_daily_7d (T1))
```

Q6 – Audit query: requests by a specific API key between two timestamps

List all requests made by a specific API key between two timestamps.

```
Result ← σ(key_id = :key_id
           AND requested_at ≥ :ts_start
           AND requested_at ≤ :ts_end) (Request)
```

Q7 – Duplicate idempotency_key detection within a project

Find duplicate idempotency_key values within a project (potential replay/retry duplicates).

```
T1 ← γ project_id, idempotency_key; COUNT(*)→cnt
        (σ idempotency_key IS NOT NULL (Request))
Result ← σ cnt > 1 (T1)
```

Q8 – Projects using more than 80% of monthly quota

List projects that have used more than 80% of their monthly quota.

```
T1 ← σ(billing_month = :current_month) (MonthlyQuota)
T2 ← π project_id, billing_month, token_limit, tokens_used,
      (tokens_used / token_limit * 100)→usage_pct (T1)
Result ← σ usage_pct > 80 (T2)
```

Q9 – Top 10 API keys by rate-limit / denied events (last 30 days)

Top 10 API keys with the most rate-limit/denied events in the last 30 days.

```
T1 ← σ(denied_at ≥ NOW()-30days) (DeniedEvent)
T2 ← T1 ⋈(T1.request_id = Request.request_id) Request
T3 ← γ key_id; COUNT(*)→denied_count (T2)
Result ← π key_id, denied_count (TOP 10 ORDER BY denied_count DESC (T3))
```

Q10 – Projects with high-frequency calls after crossing 90% quota

Identify projects that continue high-frequency calls after crossing a 90% quota threshold.

```
Exceeded ← σ(tokens_used / token_limit > 0.90) (MonthlyQuota)
T1 ← π project_id (Exceeded)
T2 ← γ project_id; COUNT(*)→calls_after
      (T1 ⋈ (T1.project_id = Request.project_id) Request)
Result ← σ calls_after > 100 (T2)
```

Q11 – Usage and cost comparison across model versions

Compare usage and cost across versions of the same model (by provider/name/version).

```
T1 ← γ model_id; COUNT(*)→req_count, SUM(computed_cost)→total_cost (Request)
T2 ← T1 ⋈ (T1.model_id = LlmModel.model_id) LlmModel
Result ← γ provider, model_name;
          version, req_count, total_cost (T2)
```

Q12 – Projects with unusually high output/input token ratio

Find projects with unusually high output_tokens / input_tokens ratio.

```
T1 ← Request ⋈ (Request.request_id = Response.request_id) Response
T2 ← γ project_id; AVG(output_tokens / input_tokens)→avg_ratio (T1)
T3 ← γ ; AVG(output_tokens / input_tokens)→global_avg (T1)
T4 ← T2 × T3
Result ← π project_id, avg_ratio (σ avg_ratio > 3 * global_avg (T4))
```

Q13 – 100 slowest requests with model, project, and latency

List the 100 slowest requests with their model, project, and latency_ms.

```
T1 ← Request ⋈ (Request.request_id = Response.request_id) Response
T2 ← T1 ⋈ (T1.model_id = LlmModel.model_id) LlmModel
T3 ← T2 ⋈ (T2.project_id = Project.project_id) Project
Result ← π request_id, project.name, model_name, latency_ms
          (TOP 100 ORDER BY latency_ms DESC (T3))
```

Q14 – Cost and failure rate comparison between environments

Compare cost and failure rate between environments (dev vs prod).

```
T1 ← Request ⋈ (Request.request_id = Response.request_id) Response
Result ← γ environment;
          SUM(computed_cost)→total_cost,
          SUM(status='failed')/COUNT(*)→failure_rate,
          COUNT(*)→total_requests (T1)
```

Q15 – Requests with no matching response (missing response anomaly)

Find requests that have no matching response record.

```
AllReq ← π request_id (Request)
```

```
HasResp ← π request_id (Response)
Result ← AllReq - HasResp
```

Q16 – Cost attribution by API key within a project (current month)

Split total cost by API key within the same project for the current month.

```
T1 ← σ (DATE_FORMAT(requested_at, '%Y-%m') = :current_month) (Request)
T2 ← T1 ⋈ (T1.project_id = :project_id) T1
Result ← γ key_id; SUM(computed_cost)→key_cost (T2)
```

Q17 – Daily cost time series for a tenant or project (past 90 days)

Daily cost time series for a given tenant or project for the past 90 days.

```
T1 ← σ (requested_at ≥ NOW() - 90days) (Request)
T2 ← T1 ⋈ (T1.project_id = Project.project_id) Project
T3 ← σ (Project.tenant_id = :tenant_id) (T2) -- or project_id = :project_id
Result ← γ DATE(requested_at)→day; SUM(computed_cost)→daily_cost (T3)
```

Q18 – Projects with an invoice but zero usage (data inconsistency)

Identify projects where an invoice exists for the month but usage is zero.

```
T1 ← σ (billing_month = :billing_month) (Invoice)
T2 ← γ project_id; SUM(computed_cost)→actual_cost
      (σ (DATE_FORMAT(requested_at, '%Y-%m') = :billing_month) (Request))
T3 ← T1 ⋈ (T1.project_id = T2.project_id) T2
Result ← π project_id, billing_month (σ actual_cost = 0 (T3))
```

Q19 – Compliance: requests made by revoked API keys after revocation

Compliance check: list any requests made by revoked API keys after revoked_at time.

```
Revoked ← σ (status = 'revoked') (ApiKey)
T1 ← Request ⋈ (Request.key_id = Revoked.key_id) Revoked
Result ← σ (requested_at > revoked_at) (T1)
```

Q20 – Most expensive tenants and contributing models

Identify the most expensive tenants and the models contributing most to their spending.

```
T1 ← Request ⋈ (Request.project_id = Project.project_id) Project
T2 ← T1 ⋈ (T1.tenant_id = Tenant.tenant_id) Tenant
T3 ← T2 ⋈ (T2.model_id = LlmModel.model_id) LlmModel
T4 ← γ tenant_id, model_id;
      SUM(computed_cost)→cost_by_model (T3)
T5 ← γ tenant_id; SUM(computed_cost)→total_tenant_cost (T3)
T6 ← T4 ⋈ (T4.tenant_id = T5.tenant_id) T5
Result ← π tenant_id, model_id, cost_by_model, total_tenant_cost
      (TOP 20 ORDER BY total_tenant_cost DESC, cost_by_model DESC (T6))
```


Conclusion

Phase 2 of this project delivered the complete conceptual and logical database design for the Multi-Tenant LLM API Gateway system. Below we summarize key learnings and areas for improvement.

What We Learned

- EER modeling forces precision: identifying all entity types, relationship cardinalities, and weak entities up front revealed several design ambiguities in our Phase 1 assumptions (e.g., whether DeniedEvent is a separate entity or an attribute of Request). EER modeling helped us resolve these systematically.
- Weak entities model lifecycle dependencies cleanly: Response and DeniedEvent are naturally weak because they cannot exist without a parent Request. Using identifying relationships and double-bordered diamonds made this dependency explicit and ensures cascaded deletes.
- Normalization trade-offs: We deliberately denormalized project_id into Request for query performance. Recognizing where to break strict normalization (3NF/BCNF) for practical performance is an important design skill.
- Relational algebra reveals schema gaps: Attempting to write algebra expressions for Q15 (missing response anomaly) and Q19 (revoked key compliance) confirmed that without a proper UNIQUE constraint on Response.request_id and an indexed revoked_at on ApiKey, those queries would be either incorrect or prohibitively slow.
- Candidate keys matter for business rules: Discovering candidate keys like (project_id, billing_month) on MonthlyQuota and Invoice directly encodes the business rule 'one quota record per project per month,' which prevents double-billing errors.

What Is Needed for Better Output

- Full MySQL DDL: The next phase should translate the relational schema into CREATE TABLE statements with all constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT), indexes on frequently-queried columns (requested_at, project_id, key_id), and appropriate storage engines (InnoDB for transactional support).
- Transactional quota enforcement: The UPDATE MonthlyQuota SET tokens_used = tokens_used + :n WHERE quota_id = :q must be wrapped in a SELECT ... FOR UPDATE transaction to prevent race conditions under concurrent requests.
- Validated SQL queries: Each of the 20 relational algebra expressions should be translated into and tested against a populated MySQL instance to verify correctness and performance.
- Refined EER diagram: Adding more attribute details (data types, null constraints) directly in the EER diagram would improve readability for future team members.
- Physical design: Partitioning the Request and Response tables by requested_at month would significantly improve analytical query performance at scale.

— End of Phase 2 Report —